ECE 562: Deep Reinforcement Learning for Robotics

Fall 2026

Assignment 2: Proximal Policy Optimization (PPO)

Instructor:	Jun Jin (`jun.jin@ualberta.ca`)
TA:	Arya Ebrahimi (`ebrahim5@ualberta.ca`)

Created on:	September 16, 2026
Updated on:	N/A
Deadline:	September 29, 2026, at 11:59 p.m. (Edmonton time)

This assignment explores policy optimization for continuous robotic control, focusing on the theoretical derivations and practical implementation of Proximal Policy Optimization (PPO). The assignment is structured into two main sections:

- **Theoretical Foundations of PPO (40 points):** Mathematical derivation and bias-variance spectrum of Generalized Advantage Estimation (GAE), and formal analysis of the clipped surrogate objective as a pessimistic lower bound.
- **Continuous Control Implementation in MuJoCo (60 points):** Implementation of the core algorithmic components of PPO (continuous Gaussian policy distribution, rollout buffer with GAE, clipped surrogate policy loss, and value network updates) using a single standard `InvertedPendulum-v5` environment in MuJoCo, followed by empirical ablation studies.

To earn full credit, clearly define all notation and assumptions, show the important intermediate steps in your derivations, and provide clear justifications for each empirical finding.

Unless otherwise stated, use the following assumptions:

- The state space $\mathcal{S} \subseteq \mathbb{R}^d$ and action space $\mathcal{A} \subseteq \mathbb{R}^m$ are continuous.
- Rewards are bounded, and the discount factor satisfies $0 \leq \gamma < 1$.
- The stochastic policy $\pi_\theta(a \mid s)$ parameterizes an independent multivariate Gaussian distribution with mean vector $\mu_\theta(s)$ and diagonal covariance matrix $\Sigma = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_m^2)$.

Submission Instructions Submit your completed assignment through Canvas containing:

- **`report.pdf`:** Your solutions to the theoretical derivations in Section 1 and your empirical evaluation plots, ablation figures, and written analysis for Section 2. You may prepare this report in LaTeX or write it neatly by hand and scan it into a clear PDF.
- **`code/`:** The codebase containing your completed files (`actor_critic.py`, `buffer.py`, and `ppo.py`) with all marked `TODO` sections implemented. Your code must run with the provided training harness `train.py` without runtime errors.

1 Theoretical Foundations of PPO (40 points)

This section examines Generalized Advantage Estimation (GAE) and the clipped surrogate objective.

Q1 **(20 points) Generalized Advantage Estimation (GAE) and Bias-Variance Trade-off**

Let $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$ denote the standard 1-step temporal difference (TD) residual for an approximate state-value function V. For any horizon $k \geq 1$, define the k-step advantage estimator as the sum of k discounted TD residuals:

$$\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k V(s_{t+k}).$$

Generalized Advantage Estimation defines $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$ as the exponentially-weighted average of all k-step estimators:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda) \sum_{k=1}^{\infty} \lambda^{k-1} \hat{A}_t^{(k)}, \qquad \text{where } \lambda \in [0,1].$$

(a) **(8 points)** Prove by algebraic substitution and grouping powers of γ and λ that this infinite mixture contracts directly to:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V.$$

(b) **(5 points)** Consider the setting $\lambda = 0$. Show that $\hat{A}_t^{\mathrm{GAE}(\gamma,0)} = \delta_t^V$. Under what condition on V is this estimator an unbiased estimate of the true advantage $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$?

(c) **(7 points)** Consider the setting $\lambda = 1$. Show that for infinite-horizon rollouts generated under policy π,

$$\mathbb{E}_{\tau\sim\pi}\big[\hat{A}_t^{\mathrm{GAE}(\gamma,1)} \,\big|\, s_t, a_t\big] = A^\pi(s_t, a_t) + \big(V^\pi(s_t) - V(s_t)\big),$$

and use this to show that the resulting policy gradient estimate is unbiased,

$$\mathbb{E}\big[\hat{A}_t^{\mathrm{GAE}(\gamma,1)} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big] = \mathbb{E}\big[A^\pi(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big],$$

regardless of whether the baseline value function V equals the true value function V^π. Explain why $\lambda = 1$ often leads to excessively noisy policy gradients in continuous robotic tasks.

Q2 **(20 points) The Clipped Surrogate Objective as a Pessimistic Lower Bound**

Let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$ denote the importance sampling ratio. The PPO clipped surrogate objective is defined as:

$$L^{\mathrm{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \Big) \right],$$

where $\epsilon \in (0,1)$ is the clipping hyperparameter (e.g., $\epsilon = 0.2$).

(a) **(10 points)** Analyze the objective across the four operational quadrants determined by the sign of $\hat{A}_t$ and the value of $r_t(\theta)$ relative to the clipping bounds. For each case, state the exact value of the term inside the expectation and whether the gradient $\nabla_\theta L^{\mathrm{CLIP}}(\theta)$ is non-zero or clipped to zero:

i. $\hat{A}_t > 0$ and $r_t(\theta) > 1+\epsilon$
ii. $\hat{A}_t > 0$ and $r_t(\theta) \leq 1+\epsilon$
iii. $\hat{A}_t < 0$ and $r_t(\theta) < 1-\epsilon$
iv. $\hat{A}_t < 0$ and $r_t(\theta) \geq 1-\epsilon$

(b) **(10 points)** Prove that $L^{\mathrm{CLIP}}(\theta) \leq r_t(\theta)\hat{A}_t$ holds for all $r_t(\theta) \geq 0$ and $\hat{A}_t \in \mathbb{R}$. Explain why this makes L^{CLIP} a *pessimistic lower bound* on the unclipped surrogate objective.

2 Continuous Control Implementation in MuJoCo (60 points)

In this section, you will implement the core algorithmic components of PPO in PyTorch and train your policy on a single continuous control robotic environment (`InvertedPendulum-v5`) in MuJoCo.

The outer iteration loop, standard environment rollout collection, evaluation protocol, and metric logging are fully implemented for you in `train.py`. You only need to complete the four marked `TODO` sections across three modular files:

- `actor_critic.py`: Neural network definitions for the actor and critic networks (`TODO 1`).
- `buffer.py`: Rollout storage and 1D sequential GAE calculation (`TODO 2`).
- `ppo.py`: Mini-batch advantage normalization, clipped surrogate loss, value loss, and gradient clipping (`TODO 3` and `TODO 4`).

Q3 **(15 points) Continuous Action Distribution (`actor_critic.py`)**

In `actor_critic.py`, complete `TODO 1` inside `ActorCritic.get_action_and_value`:

(a) **(8 points)** Given the mean vector $\mu_\theta(s)$ and standard deviations $\sigma = \exp(\log \sigma)$, construct an independent Gaussian distribution using `torch.distributions.Normal`. For a given or sampled action tensor of shape `(batch_size, act_dim)`, compute the joint log-likelihood:

$$\log \pi_\theta(a \mid s) = \sum_{i=1}^{m} \log \pi_\theta(a_i \mid s).$$

Ensure your returned `log_prob` tensor has shape `(batch_size,)`.

(b) **(7 points)** Compute the analytical joint entropy of the distribution, summed across all action dimensions, returning an entropy tensor of shape `(batch_size,)`.

Q4 **(15 points) Rollout Buffer and Sequential GAE Recursion (`buffer.py`)**

In `buffer.py`, complete `TODO 2` inside `RolloutBuffer.compute_gae`. The buffer collects transitions sequentially from a single environment instance over an iteration horizon of T steps:

(a) **(10 points)** Implement the backward recursion for $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$ iterating backwards in time from step $t = T-1$ down to $t = 0$:

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_{t+1}) - V(s_t),$$

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_{t+1})\hat{A}_{t+1},$$

where $d_{t+1} \in \{0, 1\}$ is the episode termination flag for transition $t+1$. Properly handle the boundary condition at the end of the rollout ($t = T-1$) using the bootstrap value $V(s_T)$ and done flag d_T.

(b) **(5 points)** Compute the target returns $R_t = \hat{A}_t + V(s_t)$ for each time step $t \in \{0, \ldots, T-1\}$ and populate both the advantage and return buffers.

Q5 **(15 points) Clipped Surrogate Loss and Parameter Updates (`ppo.py`)**

In `ppo.py`, complete `TODO 3` and `TODO 4` inside `PPOAgent.update`:

(a) **(4 points)** Implement mini-batch advantage normalization:

$$\hat{A}_{\text{norm}} = \frac{\hat{A} - \mu_{\hat{A}}}{\sigma_{\hat{A}} + 10^{-8}}.$$

(b) **(6 points)** Compute the probability ratio $r_t(\theta) = \exp\left(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)\right)$ and the clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}\left[\min\left(r_t(\theta)\hat{A}_{\text{norm}},\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_{\text{norm}}\right)\right].$$

(c) **(5 points)** Implement the mean squared error value function loss

$$L^{\text{VF}}(\phi) = \frac{1}{2}\hat{\mathbb{E}}\left[(V_\phi(s_t) - R_t)^2\right]$$

and total objective $L = -L^{\text{CLIP}} + c_1 L^{\text{VF}} - c_2\mathcal{H}$. In `TODO 4`, execute the backward pass and apply gradient clipping using `torch.nn.utils.clip_grad_norm_` with maximum threshold $g_{\max} = 0.5$.

Q6 **(15 points) Empirical Evaluation and Ablation Study**

Train your completed PPO implementation on the standard MuJoCo `InvertedPendulum-v5` environment for 1.2×10^5 environment interaction steps across three random seeds ($\{1, 2, 3\}$).

(a) **(5 points)** Plot the evaluation episodic return (mean $\pm$ one standard deviation shaded region) versus total environment steps for the default hyperparameters ($\epsilon = 0.2, \lambda = 0.95, \gamma = 0.99$). Verify that your policy successfully balances the pendulum at maximum return (1000).

(b) **(5 points) GAE Trade-off Ablation:** Fix all other parameters and train policies with $\lambda \in \{0.0, 0.95, 1.0\}$. Plot the learning curves on a single graph. Explain the empirical differences in convergence speed, variance, and final performance.

(c) **(5 points) Policy Clipping Ablation:** Train an unclipped variant of your policy ($\epsilon = \infty$). Plot the learning curve and tracked approximate KL divergence. Describe what failure modes emerge when clipping is disabled in continuous action spaces.